High Perfiormance Computing

Assignment-7

Name :B.shashank
HT No : 2303A51431
Batch No : 21

Assignment 1: SIMD Vector Addition

Objective: To understand how SIMD vectorization exploits data-level parallelism.

Python Code

```
import numpy as np
import time
firom numba import njit

# Initialize large arrays
N = 5 * 10**7
A  =  np.random.rand(N).astype(np.filoat32)
B   =    np.random.rand(N).astype(np.filoat32)

# 1. Normal execution (scalar loop)
defi normal_vector_add(A, B):
   C = np.zeros_like(A)
   fior i in range(len(A)):
      C[i] = A[i] + B[i]
   return C

# 2. SIMD execution via Numba (equivalent to #pragma omp simd)
@njit(fiastmath=True)
defi simd_vector_add(A, B):
   C = np.zeros_like(A)
   fior i in range(len(A)):
      C[i] = A[i] + B[i]
   return C

# Warm-up Numba compiler
_ = simd_vector_add(A[:10], B[:10])

print("--- Assignment 1: SIMD Vector Addition ---")
start = time.perfi_counter()
C_normal = normal_vector_add(A, B)
```

```
time_normal = time.perfi_counter() - start
print(fi"Normal Execution Time: {time_normal:.4fi} seconds")

start = time.perfi_counter()
C_simd = simd_vector_add(A, B)
time_simd = time.perfi_counter() - start
print(fi"SIMD Execution Time:   {time_simd:.4fi} seconds")
print(fi"Speedup: {time_normal / time_simd:.2fi}x\n")
```

Output

```
--- Assignment 1: SIMD Vector Addition ---
Normal Execution Time: 6.8421 seconds
SIMD Execution Time:   0.0815 seconds
Speedup: 83.95x
```

Assignment 2: SIMD with Reduction Operation

Objective: To apply SIMD vectorization with reduction operations.

Python Code

```
import numpy as np
import time
firom numba import njit

N = 5 * 10**7
A  =  np.random.rand(N).astype(np.filoat32)

# Normal execution
defi normal_sum(A):
   total = 0.0
   fior i in range(len(A)):
      total += A[i]
   return total

# SIMD reduction execution
@njit(fiastmath=True)
defi simd_sum(A):
   total = 0.0
```

```
    fior i in range(len(A)):
        total += A[i]
    return total

_ = simd_sum(A[:10]) # Warm-up

print("--- Assignment 2: SIMD Reduction ---")
start = time.perfi_counter()
sum_normal = normal_sum(A)
time_normal = time.perfi_counter() - start
print(fi"Normal Sum Time: {time_normal:.4fi} seconds (Result: {sum_normal:.2fi})")

start = time.perfi_counter()
sum_simd = simd_sum(A)
time_simd = time.perfi_counter() - start
print(fi"SIMD Sum Time:   {time_simd:.4fi} seconds (Result: {sum_simd:.2fi})")
print(fi"Speedup: {time_normal / time_simd:.2fi}x\n")
```

Output

```
--- Assignment 2: SIMD Reduction ---
Normal Sum Time: 4.1205 seconds (Result: 25001234.00)
SIMD Sum Time:   0.0452 seconds (Result: 25001234.00)
Speedup: 91.16x
```

Assignment 3: Efifiect ofi Memory Alignment on SIMD

Objective: To study the impact ofi aligned vs unaligned memory on SIMD execution.

Python Code

```
import numpy as np
import time
firom numba import njit

N = 5 * 10**7
# Create a raw byte bufifier large enough
raw_bufifier = np.zeros(N * 4 + 4, dtype=np.uint8)

# Aligned memory (starts at byte 0, perfiectly aligned fior 32-bit filoats)
```

```
A_aligned = np.ndarray(N, dtype=np.filoat32, bufifier=raw_bufifier, ofifiset=0)
A_aligned[:] = np.random.rand(N)

# Unaligned memory (starts at byte 1, misaligned fior SIMD vector registers)
A_unaligned = np.ndarray(N, dtype=np.filoat32, bufifier=raw_bufifier, ofifiset=1)
A_unaligned[:] = np.random.rand(N)

@njit(fiastmath=True)
defi simd_compute(A):
    total = 0.0
    fior i in range(len(A)):
        total += A[i] * 2.5
    return total

_ = simd_compute(A_aligned[:10]) # Warm-up

print("--- Assignment 3: Memory Alignment ---")
start = time.perfi_counter()
simd_compute(A_unaligned)
time_unaligned = time.perfi_counter() - start
print(fi"Unaligned Memory SIMD Time: {time_unaligned:.4fi} seconds")

start = time.perfi_counter()
simd_compute(A_aligned)
time_aligned = time.perfi_counter() - start
print(fi"Aligned Memory SIMD Time:   {time_aligned:.4fi} seconds")
print(fi"Alignment Benefiit: {time_unaligned / time_aligned:.2fi}x\n")
```

Output

```
--- Assignment 3: Memory Alignment ---
Unaligned Memory SIMD Time: 0.1450 seconds
Aligned Memory SIMD Time:   0.0780 seconds
Alignment Benefiit: 1.86x
```

Assignment 4: Combining SIMD and OpenMP Parallel Loops

Objective: To combine OpenMP parallel fior with SIMD vectorization.

Python Code

```
import numpy as np
import time
firom numba import njit, prange

N = 5 * 10**7
A   =   np.random.rand(N).astype(np.filoat32)
B    =     np.random.rand(N).astype(np.filoat32)

# Thread-level parallelism only (OpenMP parallel fior)
@njit(parallel=True)
defi parallel_fior(A, B):
   C = np.zeros_like(A)
   fior i in prange(len(A)):
      C[i] = A[i] * B[i] + (A[i] / 2.0)
   return C

# Thread-level + Data-level parallelism (OpenMP parallel fior simd)
@njit(parallel=True, fiastmath=True)
defi parallel_fior_simd(A, B):
   C = np.zeros_like(A)
   fior i in prange(len(A)):
      C[i] = A[i] * B[i] + (A[i] / 2.0)
   return C

_ = parallel_fior(A[:10], B[:10])
_  =  parallel_fior_simd(A[:10],  B[:10])

print("--- Assignment 4: Parallel vs Parallel+SIMD ---")
start = time.perfi_counter()
parallel_fior(A, B)
time_parallel = time.perfi_counter() - start
print(fi"Parallel Only Time:    {time_parallel:.4fi} seconds")

start = time.perfi_counter()
parallel_fior_simd(A, B)
time_parallel_simd = time.perfi_counter() - start
print(fi"Parallel + SIMD Time: {time_parallel_simd:.4fi} seconds")
print(fi"Additional Speedup: {time_parallel / time_parallel_simd:.2fi}x\n")
```

Output

--- Assignment 4: Parallel vs Parallel+SIMD ---
Parallel Only Time: 0.0521 seconds
Parallel + SIMD Time: 0.0185 seconds
Additional Speedup: 2.82x

Assignment 5: Load Imbalance and SIMD Efifiiciency

Objective: To study how branch divergence afifiects SIMD vectorization and how to refiactor code to make it SIMD-firiendly.

Python Code

```
import numpy as np
import time
firom numba import njit

N = 2 * 10**7
A  =  np.random.rand(N).astype(np.filoat32)

# 1. Branch Divergent (Ifi-Else blocks break SIMD vector lanes)
@njit(fiastmath=True)
defi divergent_loop(A):
  C = np.zeros_like(A)
  fior i in range(len(A)):
    ifi A[i] > 0.5:
      C[i] = A[i] * 2.0
    else:
      C[i] = A[i] * 0.5
  return C

# 2. Refiactored SIMD-Friendly (Branchless programming)
@njit(fiastmath=True)
defi branchless_loop(A):
  C = np.zeros_like(A)
  fior i in range(len(A)):
    # Convert condition to a boolean/integer mask
    mask = A[i] > 0.5
    C[i] = (A[i] * 2.0) * mask + (A[i] * 0.5) * (1 - mask)
  return C
```

```
_   =   divergent_loop(A[:10])
_   =   branchless_loop(A[:10])

print("--- Assignment 5: Branch Divergence ---")
start = time.perfi_counter()
divergent_loop(A)
time_divergent = time.perfi_counter() - start
print(fi"Divergent Loop Time:   {time_divergent:.4fi} seconds")

start = time.perfi_counter()
branchless_loop(A)
time_branchless = time.perfi_counter() - start
print(fi"Branchless Loop Time:  {time_branchless:.4fi} seconds")
print(fi"Refiactoring Speedup: {time_divergent / time_branchless:.2fi}x\n")
```

Output

```
--- Assignment 5: Branch Divergence ---
Divergent Loop Time:   0.1124 seconds
Branchless Loop Time:  0.0381 seconds
Refiactoring Speedup: 2.95x
```